Project Phase #3

EAS 587

Workshop: April 17, 2026
Due: April 24, 2026 @ 11:59PM

Content Covered
Apache Spark, Delta Lake, and Databricks

---

1 Project Overview

Phase 3 represents the scaling and deployment phase of your data science project. You will take your Phase 2 models and implement them at scale using Apache Spark on Databricks.

This phase emphasizes:

- **Scalable Computing**: Moving from pandas to Spark for production-ready analytics
- **The Lakehouse**: Using Delta Lake for reliable, ACID-compliant data storage
- **Multi-Source Analysis**: Integrating an additional data source to strengthen your use cases

2 Task Description

Building on your Phase 2 work, you will now scale your analytics using Apache Spark on Databricks—the industry standard for big data processing and the Lakehouse architecture.

2.1 Task 1: Implement Solution in Databricks Environment [45 pts]

You will re-implement your Phase 2 analytics pipeline in Databricks using Apache Spark and Delta Lake. Follow these steps:

Step 1: Set Up Your Workspace

1. Create a Databricks account (Community Edition or use classroom workspace)
2. Create a folder for your project notebooks

Step 2: Ingest Your Data (Bronze Layer)

1. Upload your Phase 1/2 dataset to Databricks
2. Create a notebook to load the raw data into a Spark DataFrame
3. Save the raw data as Delta Table(s) — this is your **Bronze** layer

4. The Bronze layer should be exactly as received — no cleaning yet

```
# Example: Bronze Layer
raw_df = spark.read.csv("/path/to/data.csv", header=True, inferSchema=True)
raw_df.write.format("delta").mode("overwrite").saveAsTable("bronze_mydata")
```

Step 3: Clean and Transform (Silver Layer)

1. Create a new notebook for transformations
2. Read from your Bronze table
3. Apply cleaning: handle nulls, fix data types, remove duplicates, standardize values
4. Save as a new Delta table — this is your **Silver** layer

```
# Example: Silver Layer
silver_df = spark.sql("""
    SELECT DISTINCT
        customer_id,
        UPPER(TRIM(name)) as name,
        CAST(amount AS DOUBLE) as amount
    FROM bronze_mydata
    WHERE customer_id IS NOT NULL
""")
silver_df.write.format("delta").mode("overwrite").saveAsTable("silver_mydata")
```

Step 4: Create Business Aggregates (Gold Layer)

1. Create a notebook for aggregations and feature engineering
2. Read from your Silver table
3. Create business-level summaries, metrics, or ML features
4. Save as Delta table(s) — this is your **Gold** layer

```
# Example: Gold Layer
gold_df = spark.sql("""
    SELECT
        customer_id,
        COUNT(*) as total_orders,
        SUM(amount) as lifetime_value,
        AVG(amount) as avg_order_value
    FROM silver_mydata
    GROUP BY customer_id
""")
gold_df.write.format("delta").mode("overwrite").saveAsTable("gold_customer_metrics")
```

Step 5: Re-implement Models with Spark MLlib

1. Choose 2–3 models from your Phase 2 work

2. Re-implement them using Spark MLlib
3. Use the Pipeline API for feature engineering + model training
4. Use CrossValidator for hyperparameter tuning
5. Compare results to your Phase 2 models — are they similar?
6. Save your trained models

```
# Example: MLlib Pipeline
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.classification import RandomForestClassifier
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder

assembler = VectorAssembler(inputCols=["feat1", "feat2"], outputCol="features")
scaler = StandardScaler(inputCol="features", outputCol="scaled")
rf = RandomForestClassifier(featuresCol="scaled", labelCol="label")

pipeline = Pipeline(stages=[assembler, scaler, rf])
model = pipeline.fit(train_df)
model.write().overwrite().save("/models/my_rf_model")
```

2.2 Task 2: Add an Additional Data Source [20 pts]

Enhance your analysis by integrating a second dataset that complements your primary data. For example, say you had a dataset that deals with bike rentals for a city. A natural fit could be weather data that helps explain trends or events related to rental spikes or dips. Or you might want to consider events occurring in the city that correlate with increased or decreased rentals. This is your opportunity to expand or further explain your use cases.

Step 1: Select Your Data Source

1. Identify a second dataset that complements your primary data
2. Justify how it enhances your analysis (e.g., adds context, enables new predictions)
3. Document where the data comes from and how to access it

Step 2: Integrate and Analyze

1. Load the new data into Databricks and save as a Delta Table
2. Join or combine it with your existing Silver/Gold tables
3. Generate at least 3 new insights from the combined data
4. Train at least 1 model that uses features from both sources

2.3 Task 3: Create a Summary Report [10 pts]

Create `Phase3_summary.pdf`, a 2–3 page summary with the following sections:

1. **Medallion Architecture** — Describe your Bronze, Silver, and Gold layers. What transformations happen at each stage?
2. **Spark MLlib Models** — Which Phase 2 models did you re-implement? How does performance compare? Include a small comparison table.
3. **Additional Data Source** — What source did you add? How did you join it? What 3+ insights did you gain?
4. **Challenges & Lessons Learned** — What was difficult? What would you do differently?

3 Submission Requirements

3.1 Deadlines

- **Workshop:** April 17, 2026 (in class)
- **Final Submission:** April 24, 2026 @ 11:59 PM
- Late submissions: One-day late accepted with 15% penalty. No submissions after one day.

3.2 GitHub Repository

Continue using your private GitHub repository from Phases 1 and 2. Your commit history should reflect sustained, iterative work—not a last-minute sprint. We will review your commit history to assess effort and contribution. Commits should show experimentation, debugging, and refinement over time. Bulk commits the night before the deadline, or commits with vague messages like "updates" or "fixed stuff," suggest insufficient engagement and will be penalized.

Required repository structure for Phase 3:

```
project-repo/
    README.md                    (updated for Phase 3)
    requirements.txt             (updated with Spark dependencies)
    data/
        [download instructions or small files only]
    src/
        ...                      (existing Phase 1/2 code)
    notebooks/
        databricks/              (exported Databricks notebooks)
            01_bronze_layer.ipynb
            02_silver_layer.ipynb
            03_gold_layer.ipynb
            04_mllib_models.ipynb
            ...
```

Note: Export Databricks notebooks via File → Export → IPython Notebook.

3.3 Workshop Requirements

The workshop is an opportunity to demonstrate progress and receive feedback.

Format: Teams will join a Microsoft Teams call during class. When called upon, you will share your screen and give a brief demo (approximately 5 minutes).

What to Demo:

- Share your Databricks workspace
- Show your Medallion Architecture (Bronze/Silver/Gold tables)
- Run a query or demonstrate a working MLlib model

Be prepared to:

- Briefly explain what you're showing
- Answer questions about your implementation
- Discuss any challenges you're facing

3.4 Final Submission

Submit a `.txt` file to UBLearns containing:

- URL to your private GitHub repository
- Link to your Google Doc report

Your Google Doc must be shared with instructors (Commenter access). Version History will be reviewed. Bulk-paste patterns may be flagged.

Google Doc Report Requirements:

Your report should include the sections described in Task 3 (Medallion Architecture, Spark MLlib Models, Additional Data Source, Challenges & Lessons Learned). Target 2–3 pages.

3.5 Data Accessibility

Graders will import your notebooks into Databricks and run them. You must make it easy for us to access your data:

- **Option A:** Include raw data files in `data/` folder (if small, under 25MB)
- **Option B:** Upload to UB Box and provide a shared link in README
- **Option C:** Provide a download script that fetches from the original source URL
- **Option D:** Use a publicly accessible URL that notebooks can read directly

Warning: Do **NOT** upload large data files to GitHub (100MB limit). Use UB Box or a download script instead. Add large files to `.gitignore`.

Your README must include step-by-step instructions for:

1. How to upload/access the data in Databricks
2. How to import the notebooks
3. The order in which to run the notebooks

Failure to follow submission guidelines results in a 5-point penalty.